

Angular Handbook

Complete Notes with Examples, Spring Boot REST Integration, CRUD Flow, Login Flow, and Interview Preparation

About this handbook: It explains Angular concepts step by step with examples and connects them with Spring Boot REST APIs.

What you will learn	Outcome
Angular fundamentals	Understand components, templates, binding, directives, pipes, services, routing, forms, and HTTP.
Backend integration	Call Spring Boot REST APIs from Angular using HttpClient.
Real project flow	Build a Student Management CRUD application.
Authentication basics	Understand login, JWT token storage, guards, and interceptors.
Interview preparation	Revise important Angular questions and practical scenarios.

Recommended prerequisite: basic HTML, CSS, JavaScript fundamentals, Java, Spring Boot MVC/JSP, and REST API basics.

Table of Contents

- 1. Angular Overview
- 2. JSP vs Angular Mindset
- 3. Environment Setup
- 4. Role of npm, ng, node_modules, package.json, and Project Files
- 5. How Angular Runs: Startup and Bootstrap Flow
- 6. Angular Project Structure
- 7. TypeScript Basics for Java Students
- 8. Components
- 9. Data Binding
- 10. Directives and Control Flow
- 11. Pipes
- 12. Services and Dependency Injection
- 13. Routing
- 14. Forms
- 15. HTTP and Spring Boot API Integration
- 16. Observables and RxJS Basics
- 17. Lifecycle Hooks
- 18. Standalone Components, Modules, and Signals
- 19. CRUD Mini Project: Student Management System
- 20. Login, JWT, Guard, and Interceptor
- 21. Build and Deployment
- 22. Best Practices and Common Mistakes
- 23. Practice Tasks
- 25. Glossary

- 24. Interview Questions

1. Angular Overview

- Angular is a frontend framework used to build dynamic, scalable, single-page applications using TypeScript.
- Angular applications run in the browser and communicate with backend applications through REST APIs, usually using JSON.
- Angular provides a complete structure for components, routing, forms, validations, HTTP calls, dependency injection, and application organization.
- It is widely used for enterprise applications because it encourages modular, reusable, and maintainable code.

Simple definition: Angular is used to create the user interface. Spring Boot is used to expose APIs, handle business logic, and connect with the database.

2. JSP vs Angular Mindset

Spring Boot with JSP	Spring Boot with Angular
Controller returns JSP page	Controller returns JSON data
JSP renders HTML on the server	Angular renders HTML in the browser
Model sends data to JSP	REST API sends JSON to Angular
Form submits directly to backend URL	Angular sends HTTP request to API
Mostly page reload based	Mostly no full page reload
Session-based login is common	JWT/token-based login is common

```
// JSP-style controller
@GetMapping("/students")
public String students(Model model) {
    model.addAttribute("students", studentService.getAllStudents());
    return "student-list";
}

// Angular-style REST controller
@GetMapping("/api/students")
public List<Student> students() {
    return studentService.getAllStudents();
}
```

Key idea: In Angular projects, the backend does not return pages. It returns data. Angular receives the data and displays it on the screen.

3. Environment Setup

1. Install Node.js LTS version.
2. Verify Node.js and npm installation.

```
node -v
npm -v
```

3. Install Angular CLI globally.

```
npm install -g @angular/cli
```

4. Create a new Angular project.

```
ng new student-management-ui
cd student-management-ui
```

5. Run the Angular application.

```
ng serve
# Open: http://localhost:4200
```

Common issue: If ng command is not recognized, close and reopen the terminal, or check whether npm global path is added in the system PATH.

4. Role of npm, ng, node_modules, package.json, and Project Files

When a new Angular project is created, several tools and files are generated automatically. These files are important because Angular needs Node.js, npm packages, configuration files, source files, and build tools to run properly.

4.1 Role of Node.js

- Node.js provides the JavaScript runtime used by Angular development tools on the developer machine.
- Angular itself runs in the browser, but Angular CLI, build commands, package installation, and development server use Node.js.
- Node.js is required to run commands such as ng serve, ng build, and npm install.

```
node -v
# Shows installed Node.js version
```

4.2 Role of npm

- npm stands for Node Package Manager.
- It is used to install, update, remove, and manage project dependencies.
- In Angular, dependencies such as @angular/core, @angular/router, @angular/forms, rxjs, and TypeScript are managed through npm.
- npm reads package.json and downloads required packages into the node_modules folder.

```
npm install
npm install bootstrap
npm uninstall package-name
```

4.3 Role of ng Command

- ng is the Angular CLI command.
- It helps create projects, generate components/services, run the development server, build the project, and test the application.
- The ng command is available after installing Angular CLI globally or through local project scripts.

```
npm install -g @angular/cli
ng new student-management-ui
ng generate component students
ng generate service services/student
ng serve
ng build
```

4.4 Role of package.json

- package.json is the main dependency and script configuration file of an Angular project.
- It contains project name, version, npm scripts, dependencies, and devDependencies.

- dependencies are packages required by the application at runtime or during normal development, such as Angular packages and RxJS.
- devDependencies are packages mainly required for development/build/testing, such as Angular CLI, TypeScript, and testing tools.

```
{
  "scripts": {
    "start": "ng serve",
    "build": "ng build"
  },
  "dependencies": {
    "@angular/core": "...",
    "@angular/router": "...",
    "rxjs": "..."
  },
  "devDependencies": {
    "@angular/cli": "...",
    "typescript": "..."
  }
}
```

Common command using package.json scripts:

```
npm start
# Internally runs: ng serve
```

4.5 Role of package-lock.json

- package-lock.json stores the exact installed versions of dependencies and their sub-dependencies.
- It helps maintain the same dependency versions across different systems.
- This file should normally be committed to Git for application projects.

4.6 Role of node_modules

- node_modules is the folder where npm downloads and stores all project dependencies.
- It is usually very large because it contains Angular packages and all nested dependencies.
- This folder should not be manually edited.
- This folder should not be committed to Git. It can be recreated anytime using npm install.

```
npm install
# Recreates node_modules based on package.json and package-lock.json
```

4.7 Other Important Angular Project Files

- angular.json: Main Angular workspace configuration file. It defines build options, styles, assets, and project settings.
- tsconfig.json: TypeScript compiler configuration file.
- src/main.ts: Main entry file that starts the Angular application.
- src/index.html: Main HTML page loaded by the browser. Angular injects the application inside this page.
- src/styles.css or styles.scss: Global stylesheet for the application.
- src/assets/: Folder used to store static files such as images, icons, JSON files, and logos.
- src/app/app.component.ts: Root component of the application.
- src/app/app.component.html: HTML template of the root component.
- src/app/app.routes.ts: Routing configuration file in standalone Angular projects.
- src/app/app.config.ts: Application-level provider configuration file in standalone Angular projects.
- .gitignore: Defines files/folders that should not be committed to Git, such as node_modules and dist.
- README.md: Project documentation file.

4.8 Simple Difference Between npm and ng

- npm manages packages and dependencies.
- ng manages Angular-specific tasks such as creating components, serving the project, and building the application.

```
npm install      # Install dependencies
ng serve        # Run Angular dev server
ng build        # Build Angular project
```

5. How Angular Runs: Startup and Bootstrap Flow

When an Angular application starts, the browser does not directly open individual component HTML files. The browser loads index.html, Angular starts from main.ts, and then Angular renders the root component inside the page.

5.1 High-Level Startup Flow

```
Browser opens http://localhost:4200
↓
Angular dev server serves index.html
↓
index.html loads compiled JavaScript bundles
↓
main.ts executes
↓
bootstrapApplication(AppComponent, appConfig) runs
↓
Angular creates the root AppComponent
↓
<app-root> in index.html is replaced by AppComponent UI
↓
Router loads the matching page component inside <router-outlet>
↓
Components call services
↓
Services call Spring Boot REST APIs using HttpClient
↓
Response data is displayed in the component template
```

5.2 Step 1: Browser Loads index.html

The browser first loads src/index.html. This file contains the root selector where Angular will insert the application.

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <title>StudentManagementUi</title>
  <base href="/">
</head>
<body>
  <app-root></app-root>
</body>
</html>
```

Here, <app-root> is not a normal HTML tag. It is the selector of the root Angular component.

5.3 Step 2: main.ts Starts Angular

main.ts is the actual starting point of the Angular application. In modern standalone Angular projects, it usually bootstraps AppComponent like this:

```
import { bootstrapApplication } from '@angular/platform-browser';
import { appConfig } from './app/app.config';
import { AppComponent } from './app/app.component';

bootstrapApplication(AppComponent, appConfig)
  .catch((err) => console.error(err));
```

- bootstrapApplication() starts Angular.
- AppComponent is the root component.
- appConfig contains application-level providers such as router, HttpClient, interceptors, and other configurations.

5.4 Step 3: AppComponent Loads into app-root

AppComponent contains a selector. This selector must match the tag used inside index.html.

```
@Component({
  selector: 'app-root',
  standalone: true,
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css']
})
export class AppComponent {}
```

Because index.html contains <app-root></app-root>, Angular places AppComponent output there.

5.5 Step 4: Router Displays Page Components

Most applications use routing. The root component usually contains a router outlet.

```
<!-- app.component.html -->
<nav>
  <a routerLink="/students">Students</a>
  <a routerLink="/add-student">Add Student</a>
</nav>

<router-outlet></router-outlet>
```

The router checks the browser URL and loads the matching component inside <router-outlet>.

```
export const routes: Routes = [
  { path: 'students', component: StudentListComponent },
  { path: 'add-student', component: AddStudentComponent },
  { path: '', redirectTo: 'students', pathMatch: 'full' }
];
```

5.6 Step 5: Components Use Services for Data

Components should mainly handle UI logic. API calling and reusable logic should be placed inside services.

```
export class StudentListComponent {
  students: any[] = [];

  constructor(private studentService: StudentService) {}

  ngOnInit() {
    this.studentService.getStudents().subscribe({
      next: (data) => this.students = data,
    });
  }
}
```

```

        error: (err) => console.log(err)
    });
}
}

```

5.7 Step 6: Service Calls Spring Boot REST API

```

@Inject({ providedIn: 'root' })
export class StudentService {
    private apiUrl = 'http://localhost:8080/api/students';

    constructor(private http: HttpClient) {}

    getStudents() {
        return this.http.get<any[]>(this.apiUrl);
    }
}

```

The Spring Boot backend returns JSON, and Angular displays that JSON in the HTML template.

5.8 Complete Angular Runtime Flow in One Line

index.html → main.ts → AppComponent → router-outlet → page component → service → HttpClient → Spring Boot API → JSON response → template display

5.9 Angular Flow Compared with JSP Flow

- In JSP, the server prepares the HTML page and sends it to the browser.
- In Angular, the browser loads Angular once, and Angular updates the UI dynamically.
- In JSP, controller usually returns a JSP page name.
- In Angular, Spring Boot controller usually returns JSON data.
- In JSP, page navigation often reloads the page.
- In Angular, router navigation changes components without full page reload.

6. Angular Project Structure

```

student-management-ui/
├── src/
│   ├── app/
│   │   ├── app.component.ts
│   │   ├── app.component.html
│   │   ├── app.component.css
│   │   └── app.routes.ts
│   ├── assets/
│   ├── environments/
│   ├── index.html
│   └── main.ts
├── angular.json
├── package.json
└── tsconfig.json

```

File/Folder	Purpose
src/app	Main application code
main.ts	Starts/bootstrap Angular application
index.html	Main HTML entry page
app.routes.ts	Application route configuration

assets	Static images/files
package.json	Dependencies and npm scripts
angular.json	Angular build and project configuration

7. TypeScript Basics for Java Students

- TypeScript is a typed superset of JavaScript.
- Angular code is mainly written in TypeScript.
- TypeScript supports classes, interfaces, access modifiers, types, generics, and decorators.
- TypeScript code is compiled into JavaScript for browser execution.

```
let studentName: string = 'Rahul';
let age: number = 22;
let isActive: boolean = true;

interface Student {
  id: number;
  name: string;
  course: string;
}

const student: Student = {
  id: 1,
  name: 'Rahul',
  course: 'Angular'
};
```

Java	TypeScript
class Student { }	class Student { }
interface Student { }	interface Student { }
String name	name: string
int age	age: number
boolean active	active: boolean
List<Student>	Student[]

8. Components

- Components are the building blocks of Angular applications.
- A component contains TypeScript logic, HTML template, and CSS style.
- Every screen or reusable UI part can be created as a component.
- Examples: login component, dashboard component, student-list component, add-student component.

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-user',
  standalone: true,
  template: '<h1>{{ name }}</h1>'
})
export class UserComponent {
```



```
name = 'Guru';  
}
```

Component selector: The selector is the custom HTML tag used to display a component, for example `<app-user></app-user>`.

9. Data Binding

Binding Type	Syntax	Use
Interpolation	{{ title }}	Display component value in HTML
Property binding	[src]="imageUrl"	Bind value to HTML property
Event binding	(click)="save()"	Call method on user action
Two-way binding	[(ngModel)]="username"	Sync input and component variable

```
export class BindingExampleComponent {  
  title = 'Student Management';  
  imageUrl = 'assets/logo.png';  
  username = '';  
  
  sayHello() {  
    alert('Hello ' + this.username);  
  }  
}  
  
<h2>{{ title }}</h2>  
<img [src]="imageUrl" alt="Logo">  
<input [(ngModel)]="username" placeholder="Enter name">  
<button (click)="sayHello()">Click</button>
```

For ngModel: Import `FormsModule` before using `[(ngModel)]`.

10. Directives and Control Flow

- Directives change the behavior or structure of HTML.
- Structural directives add or remove elements.
- Attribute directives change appearance or behavior.

Modern Angular control flow

```
@if (isLoggedIn) {  
  <h3>Welcome User</h3>  
} @else {  
  <h3>Please Login</h3>  
}  
  
@for (student of students; track student.id) {  
  <p>{{ student.name }}</p>  
}
```

Common directive examples

```
<p *ngIf="isVisible">Hello</p>  
<li *ngFor="let item of items">{{ item }}</li>
```

```
<div [ngClass]="{ 'active': isActive }">Status</div>
<div [ngStyle]="{ 'color': textColor }">Colored Text</div>
```

11. Pipes

- Pipes transform data in the template without changing the original value.
- Common pipes include uppercase, lowercase, date, currency, json, and slice.
- Custom pipes can also be created for project-specific formatting.

```
<p>{{ name | uppercase }}</p>
<p>{{ today | date:'dd-MM-yyyy' }}</p>
<p>{{ price | currency:'INR' }}</p>
<p>{{ student | json }}</p>
```

12. Services and Dependency Injection

- Services contain reusable logic such as API calls and shared data.
- Dependency Injection allows Angular to provide service objects to components.
- Services help keep components clean and maintainable.

```
import { Injectable } from '@angular/core';

@Injectable({
  providedIn: 'root'
})
export class DataService {
  getUsers() {
    return ['Amit', 'Rahul', 'Priya'];
  }
}

constructor(private dataService: DataService) {}

ngOnInit() {
  this.users = this.dataService.getUsers();
}
```

Spring Boot	Angular
@Service	@Injectable
Constructor injection	Constructor injection
Business logic and DB calls	API calls and frontend shared logic

13. Routing

- Routing is used to navigate between components without full page reload.
- routerLink is used for navigation in HTML.
- router-outlet is the placeholder where routed components are displayed.
- Route parameters are useful for edit/view pages, such as /edit-student/10.

```
import { Routes } from '@angular/router';
import { StudentListComponent } from './student-list/student-list.component';
import { AddStudentComponent } from './add-student/add-student.component';
```

```

export const routes: Routes = [
  { path: '', redirectTo: 'students', pathMatch: 'full' },
  { path: 'students', component: StudentListComponent },
  { path: 'add-student', component: AddStudentComponent },
  { path: 'edit-student/:id', component: AddStudentComponent }
];

<a routerLink="/students">Students</a>
<a routerLink="/add-student">Add Student</a>

<router-outlet></router-outlet>

```

14. Forms

Form Type	Best For
Template-driven form	Simple forms with less logic
Reactive form	Industry-level forms with strong validation and dynamic logic

Template-driven form

```

<form #studentForm="ngForm" (ngSubmit)="saveStudent()">
  <input name="name" [(ngModel)]="student.name" required>
  <button type="submit">Save</button>
</form>

```

Reactive form

```

studentForm = new FormGroup({
  name: new FormControl('', Validators.required),
  email: new FormControl('', [Validators.required, Validators.email]),
  course: new FormControl('', Validators.required)
});

saveStudent() {
  if (this.studentForm.invalid) {
    this.studentForm.markAllAsTouched();
    return;
  }
  console.log(this.studentForm.value);
}

<form [formGroup]="studentForm" (ngSubmit)="saveStudent()">
  <input type="text" formControlName="name" placeholder="Name">

  @if (studentForm.get('name')?.invalid && studentForm.get('name')?.touched) {
    <small>Name is required</small>
  }

  <input type="email" formControlName="email" placeholder="Email">
  <select formControlName="course">
    <option value="">Select Course</option>
    <option value="Java">Java</option>
    <option value="Angular">Angular</option>
  </select>

```

```
<button type="submit">Save</button>
</form>
```

15. HTTP and Spring Boot API Integration

- Angular uses HttpClient to call REST APIs.
- HTTP calls are asynchronous and return Observables.
- Spring Boot exposes API endpoints and Angular consumes them.
- Angular should never connect directly to the database.

Spring Boot API example

```
@RestController
@RequestMapping("/api/students")
@CrossOrigin(origins = "http://localhost:4200")
public class StudentController {

    @GetMapping
    public List<Student> getAllStudents() {
        return studentService.getAllStudents();
    }

    @PostMapping
    public Student saveStudent(@RequestBody Student student) {
        return studentService.saveStudent(student);
    }
}
```

Angular service example

```
import { HttpClient } from '@angular/common/http';
import { Injectable } from '@angular/core';

@Injectable({ providedIn: 'root' })
export class StudentService {
    private apiUrl = 'http://localhost:8080/api/students';

    constructor(private http: HttpClient) {}

    getStudents() {
        return this.http.get<any[]>(this.apiUrl);
    }

    saveStudent(student: any) {
        return this.http.post(this.apiUrl, student);
    }
}

ngOnInit() {
    this.studentService.getStudents().subscribe({
        next: (data) => this.students = data,
        error: (err) => console.error(err)
    });
}
```

CORS: If Angular runs on port 4200 and Spring Boot runs on port 8080, configure CORS in Spring Boot or through a gateway/proxy.

16. Observables and RxJS Basics

- Observable is used to handle asynchronous data.
- HTTP methods return Observables.
- subscribe() is used to receive data, errors, and completion.
- Common RxJS operators include map, filter, switchMap, debounceTime, catchError, and tap.

```
import { of } from 'rxjs';

of(1, 2, 3).subscribe(value => console.log(value));

this.studentService.getStudents().subscribe({
  next: data => this.students = data,
  error: error => this.errorMessage = 'Failed to load students'
});
```

17. Lifecycle Hooks

Hook	Purpose
ngOnInit()	Runs after component initialization. Commonly used for API calls.
ngOnChanges()	Runs when input properties change.
ngAfterViewInit()	Runs after component view is initialized.
ngOnDestroy()	Runs before component is destroyed. Used for cleanup.

```
export class StudentListComponent implements OnInit, OnDestroy {
  ngOnInit() {
    console.log('Component loaded');
  }

  ngOnDestroy() {
    console.log('Component destroyed');
  }
}
```

18. Standalone Components, Modules, and Signals

- Modern Angular supports standalone components, which reduce dependency on NgModules.
- Older Angular projects commonly use NgModules. Many industry projects still have module-based structure.
- Signals are a modern reactive state feature in Angular.
- For beginners, first focus on components, routing, forms, services, and HTTP. Then learn signals and advanced state management.

```
import { signal } from '@angular/core';

count = signal(0);

increase() {
  this.count.update(value => value + 1);
}
```

```
<p>Count: {{ count() }}</p>
<button (click)="increase()">Increase</button>
```

19. CRUD Mini Project: Student Management System

Page	Purpose
/login	User login page
/dashboard	After-login dashboard
/students	View all students
/add-student	Add new student
/edit-student/:id	Edit existing student
/view-student/:id	View one student detail

Feature	Angular Part	Spring Boot Part
Add student	Reactive form + POST API	POST /api/students
View students	Table + GET API	GET /api/students
Edit student	Route param + PUT API	PUT /api/students/{id}
Delete student	Button + DELETE API	DELETE /api/students/{id}
Search	Input filter or API query	GET /api/students?keyword=...

// Suggested Angular folder structure

```
app/
├── components/
│   ├── login/
│   ├── dashboard/
│   ├── student-list/
│   ├── student-form/
│   └── student-view/
├── services/
│   ├── student.service.ts
│   └── auth.service.ts
├── guards/
│   └── auth.guard.ts
├── interceptors/
│   └── auth.interceptor.ts
└── models/
    └── student.ts
```

20. Login, JWT, Guard, and Interceptor

Step	Flow
1	User enters username and password in Angular login form.
2	Angular sends login request to Spring Boot.
3	Spring Boot validates credentials.
4	Spring Boot returns JWT token.
5	Angular stores token in localStorage/sessionStorage.
6	Interceptor attaches token in Authorization header.
7	Route guard protects private pages.

```
// Authorization header format
Authorization: Bearer eyJhbGciOiJIUzI1NiJ9...

export class AuthService {
  saveToken(token: string) {
    localStorage.setItem('token', token);
  }

  getToken() {
    return localStorage.getItem('token');
  }

  logout() {
    localStorage.removeItem('token');
  }
}

// Functional interceptor example
export const authInterceptor: HttpInterceptorFn = (req, next) => {
  const token = localStorage.getItem('token');

  if (token) {
    req = req.clone({
      setHeaders: {
        Authorization: `Bearer ${token}`
      }
    });
  }

  return next(req);
};

// Basic guard idea
export const authGuard: CanActivateFn = () => {
  const token = localStorage.getItem('token');
  return !!token;
};
```

Security note: Never store passwords in browser storage. Always validate JWT on the backend.

21. Build and Deployment

- During development, run Angular with ng serve.
- For production, build Angular using ng build.
- The production build creates static files inside dist folder.
- Angular static files can be deployed on Nginx, Apache, Tomcat static folder, cloud hosting, or served separately from backend.
- Backend API URLs should be managed through environment configuration.

```
ng build
# Output folder example:
dist/student-management-ui/

export const environment = {
  production: false,
```

```
apiBaseUrl: 'http://localhost:8080/api'
};
```

22. Best Practices and Common Mistakes

Best Practice	Reason
Keep API calls inside services	Components remain clean
Use reactive forms for complex forms	Better validation and control
Use route guards for protected pages	Prevents unauthorized navigation
Use interceptors for tokens	Avoid repeating header code
Use environment files for API URLs	Easy change between local/test/prod
Handle loading and errors	Better user experience

Common Mistake	Fix
Using ngModel without FormsModule	Import FormsModule
Using reactive forms without ReactiveFormsModule	Import ReactiveFormsModule
CORS error from browser	Enable CORS in backend or proxy
API works in Postman but not Angular	Check CORS, URL, headers, and request body
Page refresh gives 404 after deployment	Configure server rewrite to index.html
Token not sent to backend	Check interceptor registration and Authorization header

23. Practice Tasks

1. Create a component that displays student name, course, and age.
2. Create a table using @for to display a list of students.
3. Create a reactive form with name, email, mobile number, gender, course, and address.
4. Add validation messages for required, email, and mobile number pattern.
5. Create routing for student list, add student, edit student, and dashboard.
6. Create a service and call GET and POST APIs.
7. Implement delete confirmation before calling DELETE API.
8. Create a login page and store token after successful login.
9. Create an interceptor to send token in every request.
10. Create a guard to protect dashboard and student pages.

24. Interview Questions

Q. What is Angular?

A. Angular is a TypeScript-based frontend framework used to build single-page applications.

Q. What is SPA?

A. SPA means Single Page Application. It loads one main page and updates content dynamically without full page reload.

Q. What is a component?

A. A component is a reusable UI block that contains template, style, and TypeScript logic.

Q. What is data binding?

A. Data binding connects component data with the HTML template.

Q. What is two-way binding?

A. Two-way binding keeps component property and input field value synchronized using [(ngModel)].

Q. What is a directive?

A. A directive changes the structure, appearance, or behavior of DOM elements.

Q. What is a pipe?

A. A pipe transforms data in the template, such as uppercase, date, or currency.

Q. What is a service?

A. A service contains reusable logic such as API calls and shared data.

Q. What is Dependency Injection?

A. Dependency Injection is a design pattern where Angular provides required service objects to a class.

Q. What is routing?

A. Routing allows navigation between components without full page reload.

Q. What is router-outlet?

A. router-outlet is a placeholder where Angular displays the routed component.

Q. What is HttpClient?

A. HttpClient is Angular service used to make HTTP requests to backend APIs.

Q. What is Observable?

A. Observable handles asynchronous data and can emit values over time.

Q. What is ngOnInit?

A. ngOnInit is a lifecycle hook used to initialize component data, commonly for API calls.

Q. What is an interceptor?

A. An interceptor modifies HTTP requests or responses globally, commonly used for JWT token headers.

Q. What is a guard?

A. A guard controls whether a route can be accessed.

Q. What is JWT?

A. JWT is a token format used for authentication and authorization between frontend and backend.

Q. Difference between template-driven and reactive forms?

A. Template-driven forms are simple and template-focused. Reactive forms are class-driven and better for complex validation.

Q. Why use environment files?

A. Environment files help manage different API URLs and settings for local, test, and production environments.

Q. Can Angular connect directly to database?

A. No. Angular should call backend APIs. Backend connects to the database.

25. Glossary

Term	Meaning
Component	Reusable UI block

Template	HTML view of a component
Service	Reusable logic class
Directive	Changes DOM behavior or structure
Pipe	Transforms display data
Routing	Navigation between components
Observable	Asynchronous data stream
HttpClient	API calling service
Guard	Route access control
Interceptor	Global HTTP request/response modification
JWT	Authentication token
SPA	Single Page Application
CORS	Browser security mechanism for cross-origin requests